PRINCESS & BROTHERS?
PROJECT DOCUMENTATION?
Updated: October 2026?
?
OVERVIEW?
Princess & Brothers is a full-stack produce storefront and operations platform. It?
is built as a TypeScript monorepo with a Next.js App Router web application, MySQL?
database, and shared database and UI workspaces.?
?
TECHNOLOGY?
- Node.js 20 or newer and npm workspaces?
- Next.js 15, React 18, and TypeScript?
- MySQL 8 using parameterized mysql2 queries?
- Tailwind CSS, Bootstrap, and custom responsive CSS?
- Vitest for unit and integration tests?
?
REPOSITORY MAP?
- frontend/app: Next.js pages, layouts, API routes, sitemap, and server actions?
- frontend/features: storefront, authentication, admin, driver, contributor,?
payment, and privacy feature code?
- frontend/shared: shared server utilities?
- db: schema, migrations, seed scripts, and database smoke tests?
- packages/db: shared typed database package?
- packages/ui: shared UI package?
- scripts: local development preflight checks?
?
LOCAL DEVELOPMENT?
Requirements: Node.js 20+, npm 10+, and MySQL 8.?
?
1. Create a MySQL database named princesbrothers and start MySQL on the host and?
port configured in DATABASE_URL (the default is 127.0.0.1:3306).?
2. Install packages from the repository root with: npm install?
3. Copy .env.example to .env and configure DATABASE_URL and AUTH_JWT_SECRET.?
AUTH_JWT_SECRET must contain at least 32 characters.?
4. Apply the database schema with: npm run db:migrate?
5. Optionally load starter catalog data with: npm run db:seed?
6. Start development with: npm run dev?
7. Open: http://localhost:3000?
?
The root development command checks that MySQL is reachable before starting.?
The web workspace may be run directly with?
"npm run dev --workspace=@princess-brothers/web", but data-backed features still?
require MySQL.?
?
IMPORTANT COMMANDS?
- npm run dev: preflight and start the web application?
- npm run build: production build?
- npm run lint: run available workspace linters?
- npm run typecheck: typecheck web, database, and UI workspaces?
- npm test: run Vitest tests?
- npm run db:migrate: apply schema and migrations?
- npm run db:seed: load initial database data?

- npm run db:smoke: exercise database connectivity and CRUD smoke checks?
?
PUBLIC ROUTES AND FEATURES?
- /: produce storefront and product/category highlights?
- /products: searchable and filterable product catalog?
- /products/{id}: product details?
- /cart and /checkout: basket and checkout?
- /login: customer log in, account creation, and Google sign-in?
- /account and /orders: customer account and order history?
- /privacy-policy and /terms-and-conditions: published policy pages?
- /sitemap.xml: generated public-route sitemap, including active catalog products?
?
The header includes account sign-in/sign-up choices, an account-management link,?
cart access, contributor/driver shortcuts, and a light/dark theme button. The theme?
choice is held in sessionStorage, so it remains for the current browser tab?
session and is not saved as a permanent preference.?
?
DRIVER PORTAL?
Drivers sign in at /driver/login and use /driver for their dashboard. The dashboard?
shows the logged-in driver’s assigned deliveries for today, recipient/address?
details, route navigation, and delivery progress. Drivers can update delivery?
status and manage their own vehicle details, service locations, and payout?
accounts. Server actions verify an active driver session and scope reads/writes?
to that driver’s database record; one driver cannot edit another driver’s routes?
or settings.?
?
OTHER PORTALS?
- /partner: contributor access and application?
- /partner/dashboard: contributor dashboard for authorized contributors?
- /admin/dashboard: administration?
- /login/admin: administrator sign-in?
?
Accountant admins can access the private income/expense ledger and reports.?
Ledger visibility is restricted to the accountant account that recorded each?
transaction. Apply the current migrations before using the finance workflow.?
?
AUTHENTICATION AND CONFIGURATION?
Authentication uses httpOnly session cookies and MySQL-backed session state.?
Storefront accounts support password and Google sign-in. Drivers use a phone?
number and PIN. Admin accounts use password and TOTP. Contributors are subject?
to verification and approval. Keep provider credentials and signing secrets in?
.env or the deployment secret manager; never commit credentials.?
?
For local Google OAuth, register this exact authorized redirect URI in the Google?
Cloud Console and set GOOGLE_REDIRECT_URI to the same value:?
http://localhost:3000/api/auth/google/callback?
?
Common configuration variables are documented in .env.example and DEPLOYMENT.md.?
Provider credentials are needed only for enabled services such as Google OAuth,?
SMS, email, or payment gateways.?
?
DEPLOYMENT AND OPERATIONS?

Use npm run build, npm run typecheck, and npm test before deployment. Configure?
production DATABASE_URL, APP_URL, AUTH_JWT_SECRET, and the enabled provider?
credentials. Set APP_URL to the public canonical origin so generated sitemap?
links use the production domain. Apply migrations before enabling features that?
depend on new schema.?
?
Do not expose private account, order, driver, contributor, or administrator data?
in public pages or the sitemap. Review DEPLOYMENT.md for environment setup and?
release guidance. Legal/privacy text is operational documentation and should be?
reviewed by qualified counsel before production use.?